Cascade Delete Relationships

What happens if we **delete** a team that has a **relationship** with heroes?

Should those heroes be **automatically deleted** too? That's called a "**cascade**", because the initial deletion causes a cascade of other deletions.

Should their `team_id` instead be set to `NULL` in the database?

Let's see how to configure that with **SQLModel**.

> **Info**
>
> This feature, including `cascade_delete`, `ondelete`, and `passive_deletes`, is available since SQLModel version `0.0.21`.

Initial Heroes and Teams

Let's say that we have these **teams** and **heroes**.

Team Table

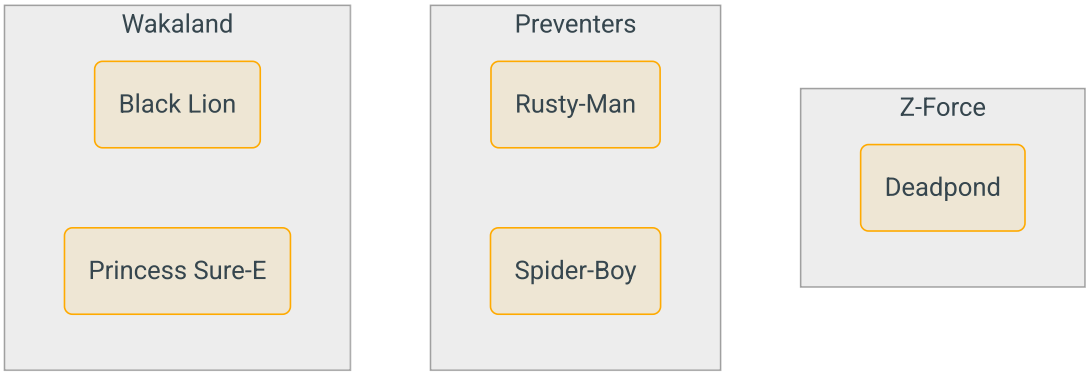
id	name	headquarters
1	Z-Force	Sister Margaret's Bar
2	Preventers	Sharp Tower
3	Wakaland	Wakaland Capital City

Hero Table

id	name	secret_name	age	team_id
1	Deadpond	Dive WIlson		1
2	Rusty-Man	Tommy Sharp	48	2
3	Spider-Boy	Pedro Parqueador		2
4	Black Lion	Trevor Challa	35	3
5	Princess Sure-E	Sure-E		3

Visual Teams and Heroes

We could visualize them like this:



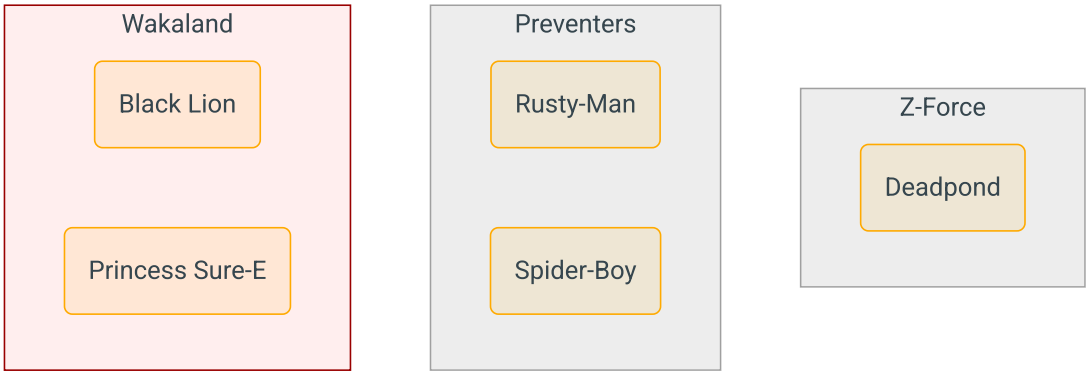
Delete a Team with Heroes

When we **delete a team**, we have to do something with the associated heroes.

By default, their foreign key pointing to the team will be set to `NULL` in the database.

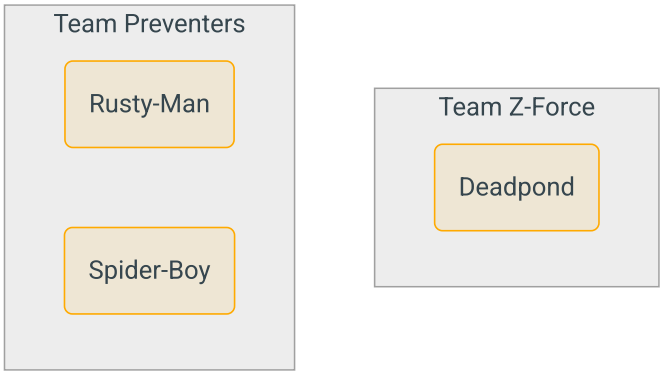
But let's say we want the associated heroes to be **automatically deleted**.

For example, we could delete the team `Wakaland`:



And we would want the heroes `Black Lion` and `Princess Sure-E` to be **automatically deleted** too.

So we would end up with these teams and heroes:



Configure Automatic Deletion

There are **two places** where this automatic deletion is configured:

- in **Python code**
- in the **database**

Delete in Python with `cascade_delete`

When creating a `Relationship()`, we can set `cascade_delete=True`.

This configures SQLModel to **automatically delete** the related records (heroes) **when the initial one is deleted** (a team).

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", cascade_delete=True)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", cascade_delete=True)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id", ondelete="CASCADE")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion not found:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E not found:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

With this configuration, when we delete a team, SQLModel (actually SQLAlchemy) will:

- Make sure the objects for the **related records are loaded**, in this case, the `heroes`. If they are not loaded, it will send a `SELECT` query to the database to get them.
- Send a `DELETE` query to the database **including each related record** (each hero).
- Finally, **delete the initial record** (the team) with another `DELETE` query.

This way, the internal **Python code** will take care of deleting the related records, by emitting the necessary SQL queries for each of them.

Tip

The `cascade_delete` parameter is set in the `Relationship()`, on the model that **doesn't have a foreign key**.

Technical Details

Setting `cascade_delete=True` in the `Relationship()` will configure SQLAlchemy to use `cascade="all, delete-orphan"`, which is the most common and useful configuration when wanting to cascade deletes.

You can read more about it in the SQLAlchemy docs.

Delete in the Database with `ondelete`

In the previous section we saw that using `cascade_delete` handles automatic deletions from the Python code.

But what happens if someone **interacts with the database directly**, not using our code, and **deletes a team with SQL**?

For those cases, we can configure the database to **automatically delete** the related records with the `ondelete` parameter in `Field()`.

`ondelete` Options

The `ondelete` parameter will set a SQL `ON DELETE` in the **foreign key column** in the database.

`ondelete` can have these values:

- `CASCADE`: **Automatically delete this record** (hero) when the related one (team) is deleted.
- `SET NULL`: Set this **foreign key** (`hero.team_id`) field to `NULL` when the related record is deleted.
- `RESTRICT`: **Prevent** the deletion of this record (hero) if there is a foreign key value by raising an error.

Set `ondelete` to `CASCADE`

If we want to configure the database to **automatically delete** the related records when the parent is deleted, we can set `ondelete="CASCADE"`.

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", cascade_delete=True)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id", ondelete="CASCADE")
    team: Team | None = Relationship(back_populates="heroes")

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", cascade_delete=True)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id", ondelete="CASCADE")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion not found:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E not found:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

Now, when we **create the tables** in the database, the `team_id` column in the `Hero` table will have an `ON DELETE CASCADE` in its definition at the database level.

This will **configure the database** to **automatically delete** the records (heroes) when the related record (team) is deleted.

Tip

The `ondelete` parameter is set in the `Field()`, on the model that **has a foreign key**.

Using `cascade_delete` or `ondelete`

At this point, you might be wondering if you should use `cascade_delete` or `ondelete`. The answer is: **both**! 🤓

The `ondelete` will **configure the database**, in case someone interacts with it directly.

But `cascade_delete` is still needed to tell SQLAlchemy that it should delete the **Python objects** in memory.

Foreign Key Constraint Support

Some databases don't support foreign key constraints.

For example, **SQLite** doesn't support them by default. They have to be manually enabled with a custom SQL command:

```
PRAGMA foreign_keys = ON;
```

So, in general it's a good idea to have both `cascade_delete` and `ondelete` configured.

> **Tip**
>
> You will learn more about how to **disable the default** automatic SQLModel (SQLAlchemy) behavior and **only rely on the database** down below, in the section about `passive_deletes`.

`cascade_delete` on `Relationship()` and `ondelete` on `Field()`

Just a note to remember... 🤓

- `ondelete` is put on the `Field()` with a **foreign key**. On the **"many"** side in "one-to-many" relationships.

```
class Hero(SQLModel, table=True):
    ...

    team_id: int = Field(foreign_key="team.id", ondelete="CASCADE")
```

- `cascade_delete` is put on the `Relationship()`. Normally on the **"one"** side in "one-to-many" relationships, the side **without a foreign key**.

```
class Team(SQLModel, table=True):
    ...

    heroes: list[Hero] = Relationship(cascade_delete=True)
```

Remove a Team and its Heroes

Now, when we **delete a team**, we don't need to do anything else, it's **automatically** going to **delete its heroes**.

Python 3.10+

```
# Code above omitted 👆

def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", cascade_delete=True)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id", ondelete="CASCADE")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion not found:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E not found:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

Confirm Heroes are Deleted

We can confirm that **after deleting the team** `Wakaland`, the heroes `Black Lion` and `Princess Sure-E` are **also deleted**.

If we try to select them from the database, we will **no longer find them**.

Python 3.10+

```python
# Code above omitted 👆

def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion not found:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E not found:", hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

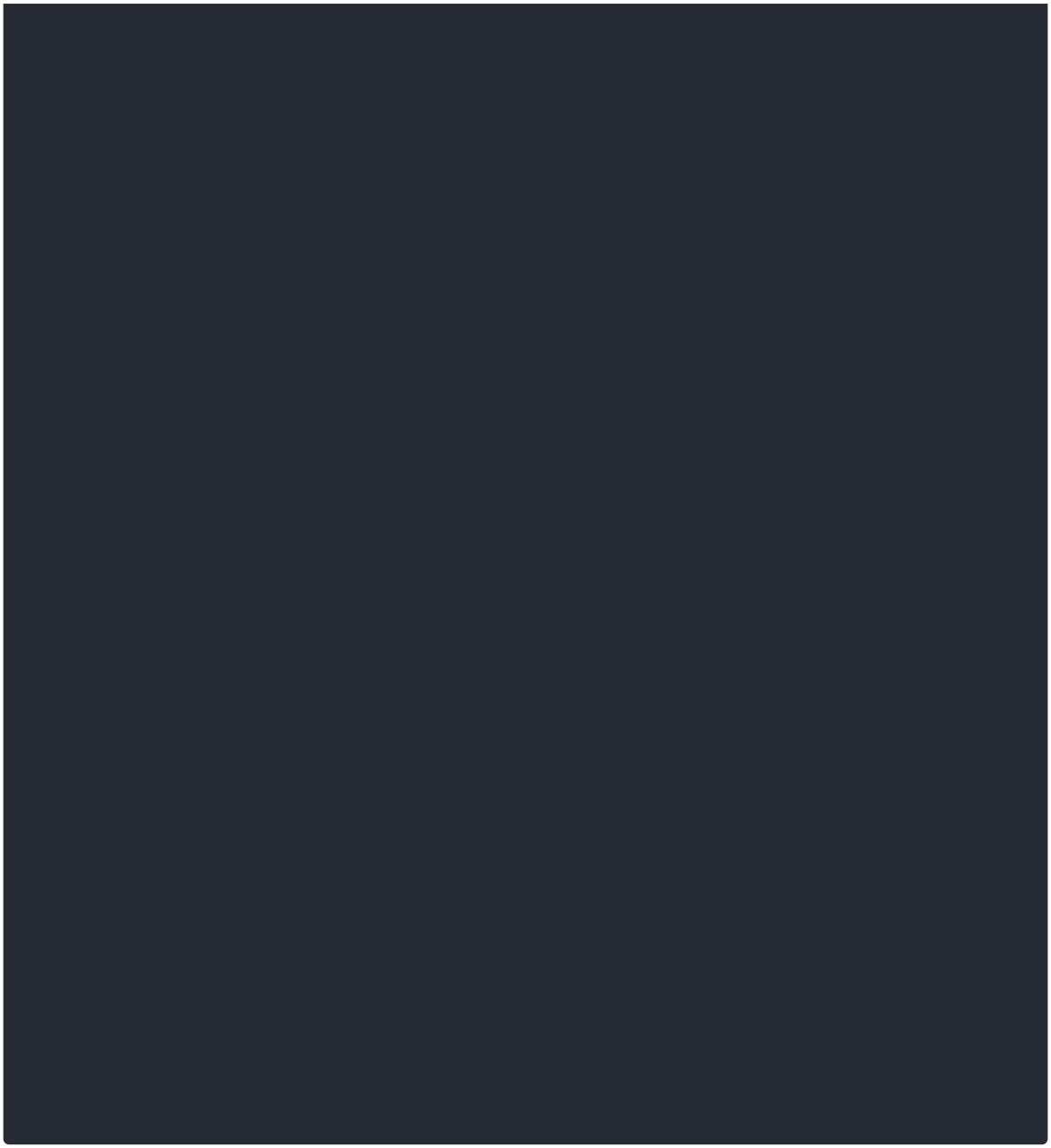
```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", cascade_delete=True)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id", ondelete="CASCADE")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion not found:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E not found:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

Run the Program with `cascade_delete=True` and `ondelete="CASCADE"`

We can confirm everything is working by running the program.

```bash
```



`ondelete` with `SET NULL`

We can configure the database to **set the foreign key** (the `team_id` in the `hero` table) to **`NULL`** when the related record (in the `team` table) is deleted.

In this case, the side with `Relationship()` won't have `cascade_delete`, but the side with `Field()` and a `foreign_key` will have `ondelete="SET NULL"`.

Python 3.10+

```
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str
```

```python
    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="SET NULL"
    )
    team: Team | None = Relationship(back_populates="heroes")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="SET NULL"
    )
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
```

```
        )
        session.add(team_wakaland)
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion has no team:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E has no team:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

The configuration above is setting the `team_id` column from the `Hero` table to have an `ON DELETE SET NULL`.

This way, when someone deletes a team from the database using SQL directly, the database will go to the heroes for that team and set `team_id` to `NULL` (if the database supports it).

Tip

The foreign key should allow `None` values (`NULL` in the database), otherwise you would end up having an Integrity Error by violating the `NOT NULL` constraint.

So `team_id` needs to have a type with `None`, like:

```
team_id: int | None
```

Not Using `ondelete="SET NULL"`

What happens if you don't use `ondelete="SET NULL"`, don't set anything on `cascade_delete`, and delete a team?

The default behavior is that SQLModel (actually SQLAlchemy) will go to the heroes and set their `team_id` to `NULL` from the **Python code**.

So, **by default**, those `team_id` fields will be **set to `NULL`**.

But if someone goes to the database and **manually deletes a team**, the heroes could end up with a `team_id` pointing to a non-existing team.

Adding the `ondelete="SET NULL"` configures the database itself to also set those fields to `NULL`.

But if you delete a team from code, by default, SQLModel (actually SQLAlchemy) will update those `team_id` fields to `NULL` even before the database `SET NULL` takes effect.

Removing a Team with `SET NULL`

Removing a team has the **same code** as before, the only thing that changes is the configuration underneath in the database.

Python 3.10+

```python
# Code above omitted 👆

def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="SET NULL"
    )
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
```

```python
        )
        session.add(team_wakaland)
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion has no team:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E has no team:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

The result would be these tables.

Team Table after `SET NULL`

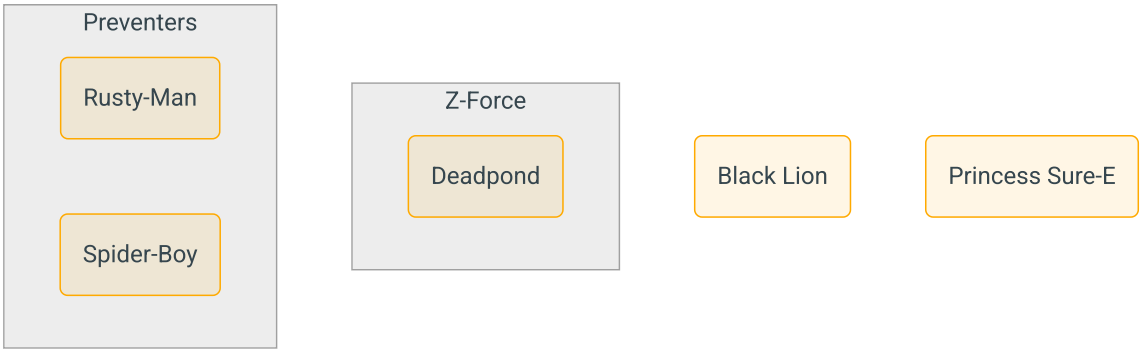
id	name	headquarters
1	Z-Force	Sister Margaret's Bar
2	Preventers	Sharp Tower

Hero Table after `SET NULL`

id	name	secret_name	age	team_id
1	Deadpond	Dive Wilson		1
2	Rusty-Man	Tommy Sharp	48	2
3	Spider-Boy	Pedro Parqueador		2
4	Black Lion	Trevor Challa	35	NULL
5	Princess Sure-E	Sure-E		NULL

Visual Teams and Heroes after `SET NULL`

We could visualize them like this:



Run the program with `SET NULL`

Let's confirm it all works by running the program now:

```bash
```

The team `Wakaland` was deleted and all of its heroes were left without a team, or in other words, with their `team_id` set to `NULL`, but still kept in the database! 🤓

Let the Database Handle it with `passive_deletes`

In the previous examples we configured `ondelete` with `CASCADE` and `SET NULL` to configure the database to handle the deletion of related records automatically. But we actually **never used that functionality** ourselves, because SQLModel (SQLAlchemy) **by default loads** the related records and **deletes** them or updates them with **NULL** before sending the `DELETE` for the team.

If you know your database would be able to correctly handle the deletes or updates on its own, just with `ondelete="CASCADE"` or `ondelete="SET NULL"`, you can use `passive_deletes="all"` in the `Relationship()` to tell SQLModel (actually SQLAlchemy) to **not delete or update** those records (for heroes) before sending the `DELETE` for the team.

Enable Foreign Key Support in SQLite

To be able to test this out with SQLite, we first need to enable foreign key support.

Python 3.10+

```python
# Code above omitted 👆

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
    with engine.connect() as connection:
        connection.execute(text("PRAGMA foreign_keys=ON"))  # for SQLite only

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select, text


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", passive_deletes="all")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="SET NULL"
    )
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
    with engine.connect() as connection:
        connection.execute(text("PRAGMA foreign_keys=ON"))  # for SQLite only


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
```

```
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion has no team:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E has no team:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

Info

You can learn more about SQLite, foreign keys, and this SQL command on the SQLAlchemy docs.

Use `passive_deletes="all"`

Now let's update the table model for `Team` to use `passive_deletes="all"` in the `Relationship()` for heroes.

We will also use `ondelete="SET NULL"` in the `Hero` model table, in the foreign key `Field()` for the `team_id` to make the database set those fields to `NULL` automatically.

Python 3.10+

```
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select, text


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", passive_deletes="all")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
```

```python
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="SET NULL"
    )
    team: Team | None = Relationship(back_populates="heroes")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select, text


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", passive_deletes="all")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="SET NULL"
    )
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
    with engine.connect() as connection:
        connection.execute(text("PRAGMA foreign_keys=ON"))  # for SQLite only


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
```

```
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion has no team:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E has no team:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

Run the Program with `passive_deletes`

Now, if we run the program, we will see that SQLModel (SQLAlchemy) is no longer loading and updating the heroes, it just sends the `DELETE` for the team.

```bash
```

`ondelete` with `RESTRICT`

We can also configure the database to **prevent the deletion** of a record (a team) if there are related records (heroes).

In this case, when someone attempts to **delete a team with heroes** in it, the database will **raise an error**.

And because this is configured in the database, it will happen even if someone **interacts with the database directly using SQL** (if the database supports it).

> **Tip**
>
> For SQLite, this also needs enabling foreign key support.

Enable Foreign Key Support in SQLite for `RESTRICT`

As `ondelete="RESTRICT"` is mainly a database-level constraint, let's enable foreign key support in SQLite first to be able to test it.

Python 3.10+

```python
# Code above omitted 👆

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
    with engine.connect() as connection:
        connection.execute(text("PRAGMA foreign_keys=ON"))  # for SQLite only

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select, text


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", passive_deletes="all")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="RESTRICT"
    )
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
    with engine.connect() as connection:
        connection.execute(text("PRAGMA foreign_keys=ON"))  # for SQLite only


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
```

```
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion has no team:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E has no team:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()


if __name__ == "__main__":
    main()
```

Use `ondelete="RESTRICT"`

Let's set `ondelete="RESTRICT"` in the foreign key `Field()` for the `team_id` in the `Hero` model table.

And in the `Team` model table, we will use `passive_deletes="all"` in the `Relationship()` for heroes, this way the default behavior of setting foreign keys from deleted models to `NULL` will be disabled, and when we try to delete a team with heroes, the database will **raise an error**.

> **Tip**
>
> Notice that we don't set `cascade_delete` in the `Team` model table.

Python 3.10+

```
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select, text


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", passive_deletes="all")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
```

```python
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="RESTRICT"
    )
    team: Team | None = Relationship(back_populates="heroes")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select, text


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", passive_deletes="all")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="RESTRICT"
    )
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
    with engine.connect() as connection:
        connection.execute(text("PRAGMA foreign_keys=ON"))  # for SQLite only


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
```

```python
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion has no team:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E has no team:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    delete_team()


if __name__ == "__main__":
    main()
```

Run the Program with `RESTRICT`, See the Error

Now, if we run the program and try to delete a team with heroes, we will see an error.

```bash
```

Great! The database didn't let us commit the mistake of deleting a team with heroes. 🤓

> **Tip**
>
> If you want to test if the `PRAGMA foreign_keys=ON` is necessary, **comment that line** and run it again, you will **not see an error**. 😱
>
> The same with `passive_deletes="all"`, if you **comment that line**, SQLModel (SQLAlchemy) will load and update the heroes before deleting the team, set their foreign key `team_id` to `NULL` and **the constraint won't work as expected**, you will not see an error. 😅

Update Heroes Before Deleting the Team

After having the `ondelete="RESTRICT"` in place, SQLite configured to support foreign keys, and `passive_deletes="all"` in the `Relationship()`, if we try to delete a team with heroes, we will see an error.

If we want to delete the team, we need to **update the heroes first** and set their `team_id` to `None` (or `NULL` in the database).

By calling the method `.clear()` from a list, we remove all its items. So, by calling `team.heroes.clear()` and saving that to the database, we disassociate the heroes from the team, that will set their `team_id` to `None`.

> **Tip**
>
> Calling `team.heroes.clear()` is very similar to what SQLModel (actually SQLAlchemy) would have done if we didn't have `passive_deletes="all"` configured.

Python 3.10+

```python
# Code above omitted 👆

def remove_team_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        team.heroes.clear()
        session.add(team)
        session.commit()
        session.refresh(team)
        print("Team with removed heroes:", team)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select, text


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team", passive_deletes="all")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(
        default=None, foreign_key="team.id", ondelete="RESTRICT"
    )
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)
    with engine.connect() as connection:
        connection.execute(text("PRAGMA foreign_keys=ON"))  # for SQLite only


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
```

```python
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)


def remove_team_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        team.heroes.clear()
        session.add(team)
        session.commit()
        session.refresh(team)
        print("Team with removed heroes:", team)


def delete_team():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Wakaland")
        team = session.exec(statement).one()
        session.delete(team)
        session.commit()
        print("Deleted team:", team)


def select_deleted_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Black Lion")
        result = session.exec(statement)
        hero = result.first()
        print("Black Lion has no team:", hero)

        statement = select(Hero).where(Hero.name == "Princess Sure-E")
        result = session.exec(statement)
        hero = result.first()
        print("Princess Sure-E has no team:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    remove_team_heroes()
    delete_team()
    select_deleted_heroes()


if __name__ == "__main__":
    main()
```

Run the Program Deleting Heroes First

Now, if we run the program and delete the heroes first, we will be able to delete the team without any issues.

```bash
```

Conclusion

In many cases, **you don't really need to configure anything**. 😎

In some cases, when you want to **cascade** the delete of a record to its related records automatically (delete a team with its heroes), you can:

- Use `cascade_delete=True` in the `Relationship()` on the side **without a foreign key**
- And use `ondelete="CASCADE"` in the `Field()` with the **foreign key**

That will **cover most of the use cases**. 🚀

And if you need something else, you can refer the additional options described above. 🤓